

Sets

CSCI 1101

2024-03-26

Data Structures Review

So far we've met two kinds of **data structures** in python:

Lists are good for storing a variable number of homogeneous items:

```
xs = [2, 4, 6, 8]      # list int
```

Tuples are good for storing a fixed number of heterogeneous items:

```
john = ("Lennon", "John", 1940) # tuple [str, str, int]
```

For both of these the *order* and *multiplicity* of elements matter:

```
[1, 2, 3] != [1, 2, 2, 3]  
(42 , True) != (True , 42)
```

Sets in Math

In math, a **set** is a collection of items.

The only predicate we have on an item and a set is whether the item is an *element* (synonym: “member”) of the set.

We write “ $x \in A$ ” to mean that the item x is an element of the set A .

There is no concept of *where* x occurs in A (order)
or *how many times* x occurs in A (multiplicity).

The only thing we can ask is whether it is *in* or *out*.

Two sets are *equal* exactly when they have the same members.

Sets in Python

Python has a data structure for sets, called `set`.

For homogeneous sets we will keep track of the element type as well.

We write a set by surrounding its elements with *braces*:

```
beatles = {"John", "Paul", "George", "Ringo"} # : set str
```

To convert a list or other iterator into a set use the `set` type converting function:

```
nums = set(range(10))
```

To create an **empty set** call the `set` function without arguments:

```
nothing = set()
```

Set Membership and Equality in Python

To test whether an item is an *element* of a set use the `_in_` operator:

```
> "Paul" in beatles  
True
```

Two sets are *equal* exactly when they have the same members:

```
> beatles == {"Ringo", "Paul", "George", "Ringo", "John"}  
True
```

Python restricts sets to contain only *immutable* (technically, *hashable*) elements.

Iterating Over Sets

Like lists, python sets are *iterable*, meaning we can use them with `for` loops:

```
def greet_all(collection) :  
    for item in collection:  
        print(f"hello {item}!")
```

When you iterate over a set you can't control the element order:

```
greet_all(beatles)
```

But you can use the `list` function to convert the set to a list:

```
greet_all(sorted(list(beatles)))
```

Adding and Removing Set Elements

To *remove* an element from a set use the `set.remove` method:

```
beatles.remove("Ringo")
```

To *add* an element to a set use the `set.add` method:

```
beatles.add("Pete")
```

To *remove all* of the elements of a set use the `set.clear` method:

```
beatles.clear()
```

To *add all* of the elements of a collection to a set use the `set.update` method:

```
beatles.update(["John", "Paul", "George", "Ringo"])
```

Set Operations in Math

In math, given two sets A and B we can form the following sets:

union: The set $A \cup B$ contains all items contained in *either* A *or* B .

intersection: The set $A \cap B$ contains all items contained in *both* A *and* B .

difference: The set $A - B$ contains all items contained in A *but not* B .

Set Operations in Python

In python, given sets:

```
(small , even) = ({0, 1, 2, 3, 4} , {0, 2, 4, 6, 8})
```

We take their:

union using the `set.union` method:

```
set.union(small , even)  
small | even      # shorthand  _|_
```

intersection using the `set.intersection` method:

```
set.intersection(small , even)  
small & even      # shorthand  _&_
```

difference using the `set.difference` method:

```
set.difference(small , even)  
small - even     # shorthand  _- _
```

Activity: Symmetric Difference

Another operations on sets is:

symmetric difference: The set $A \Delta B$ contains all elements that are contained in *exactly one of* A or B .

Write the function `symm_diff` : (set a , set a) --> set a that implements this operation in python.

For example:

```
> symm_diff ({1, 2} , {2, 3})  
{1, 3}
```

Subsets

In math, a set A is a *subset* of a set B , written “ $A \subseteq B$ ”, if every element of A is also an element of B .

In Python we ask whether one set is a subset of another with the `set.issubset` method:

```
set.issubset({1, 2} , {1, 2, 3})  
{1, 2} <= {1, 2, 3}      # shorthand  _<=_
```

We can use the other order operators `_>=_`, `_<_`, `_>_` on sets as well.

Activity: Proper Superset

A set A is a *superset* of a set B just in case B is a subset of A .

A superset or subset relation is called *proper* just in case the two sets are not equal.

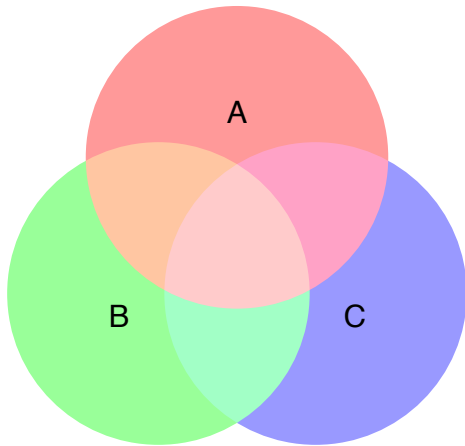
The order operator `_>_` : `(set c , set c) --> bool` represents the *proper superset* relation on sets.

Write this function in python using only `set.issubset` together with boolean operators and equality testing:

```
def proper_superset(a , b) :
```

Venn Diagrams

The regions of a **Venn diagram** represent sets that we can form from a given collection of sets under the operations of union, intersection, and difference.



Activity: Venn Diagrams

Consider the following sets:

$$A = \{1, 2, 3, 4\}$$

$$B = \{2, 4, 6, 8\}$$

$$C = \{3, 4, 5, 6\}$$

Take turns:

- picking a region in the Venn diagram for these sets,
- describing that region using set union ($\cup$), intersection ($\cap$), and difference ($-$),
- translating this description into a Python expression,
- evaluating this python expression to see what is in the set.

To Do

Finish *Homework 7: While loops and Files* on CodeRunner by 11:00AM this Thursday.